

```
1 package com.tp.jpa.repository;
2 import com.tp.jpa.model.entities.Base;
3 import com.tp.jpa.util.JPAUtil;
4 import jakarta.persistence.EntityManagerFactory;
5
6
7 import java.util.List;
8 import java.util.Optional;
9
10 public abstract class BaseRepository<T extends Base
    > {
11
12     protected final Class<T> entityClass;
13     protected final EntityManagerFactory emf;
14
15     public BaseRepository(Class<T> entityClass) {
16         this.entityClass = entityClass;
17         this.emf = JPAUtil.getEntityManagerFactory();
18     }
19
20     // ===== 1. Guardar (T entity) =====
21     public T guardar(T entity) {
22         var em = emf.createEntityManager();
23         var tx = em.getTransaction();
24
25         try {
26             tx.begin();
27             T merged = em.merge(entity);
28             tx.commit();
29             return merged;
30         } catch (Exception e) {
31             if (tx.isActive()) {
32                 tx.rollback();
33             }
34             throw new RuntimeException("Error al
guardar la entidad en BaseRepository", e);
35         } finally {
36             em.close();
37         }
38     }
39
```

```

40      // ===== 2. buscarPorId(Long id) =====
41      public Optional<T> buscarPorId(Long id) {
42          var em = emf.createEntityManager();
43
44          try {
45              T entity = em.find(entityClass, id);
46              return Optional.ofNullable(entity); //
retorna empty si es null
47
48          } finally {
49              em.close();
50          }
51      }
52
53      // ===== 3. listarActivos =====
54      public List<T> listarActivos() {
55          var em = emf.createEntityManager();
56
57          try {
58              String jpql = "SELECT e FROM " +
entityClass.getSimpleName() + " e WHERE e.eliminado
= false";
59              return em.createQuery(jpql, entityClass).
getResultList();
60
61          } finally {
62              em.close();
63          }
64      }
65
66      // ===== 3. listarInactivos =====
67      public List<T> listarInactivos() {
68          var em = emf.createEntityManager();
69
70          try {
71              String jpql = "SELECT e FROM " +
entityClass.getSimpleName() + " e WHERE e.eliminado
= true";
72              return em.createQuery(jpql, entityClass).
getResultList();
73

```

```
74         } finally {
75             em.close();
76         }
77     }
78
79
80     // ===== 4. eliminarLogico =====
81
82     public boolean eliminarLogico(Long id) {
83         var em = emf.createEntityManager();
84         var tx = em.getTransaction();
85
86         try {
87             T entity = em.find(entityClass, id);
88
89             if (entity == null) {
90                 return false; //mo existe -> retorna
91             }
92
93             tx.begin();
94             entity.setEliminado(true);
95             em.merge(entity);
96             tx.commit();
97
98             return true;
99
100         } catch (Exception e) {
101             if (tx.isActive()) {
102                 tx.rollback();
103             }
104             throw new RuntimeException("Error al
105 eliminar lógicamente la entidad", e);
106         } finally {
107             em.close();
108         }
109     }
110
111     // Alta logixa
112
```

```
113     public boolean AltaLogica(Long id) {
114         var em = emf.createEntityManager();
115         var tx = em.getTransaction();
116
117         try {
118             T entity = em.find(entityClass, id);
119
120             if (entity == null) {
121                 return false; //no existe -> retorna
122                 falso
123             }
124
125             tx.begin();
126             entity.setEliminado(false);
127             em.merge(entity);
128             tx.commit();
129
130             return true;
131         } catch (Exception e) {
132             if (tx.isActive()) {
133                 tx.rollback();
134             }
135             throw new RuntimeException("Error al dar
136             de alta la entidad la entidad", e);
137         } finally {
138             em.close();
139         }
140     }
141
142
143 }
144
```

```
1 package com.tp.jpa.repository;
2
3 import com.tp.jpa.model.entities.Producto;
4 import jakarta.persistence.EntityManager;
5
6 import java.util.List;
7
8 public class ProductoRepository extends
  BaseRepository<Producto> {
9
10     public ProductoRepository() {
11         super(Producto.class);
12     }
13
14     public List<Producto> buscarPorCategoria(Long
  categoriaId) {
15         var em = emf.createEntityManager();
16
17         try {
18             String jpql = "SELECT p FROM Producto p "
19 +
20             "WHERE p.categoria.id = :
  categoriaId " +
21             "AND p.eliminado = false";
22
23             return em.createQuery(jpql, Producto.
  class)
24                 .setParameter("categoriaId",
  categoriaId)
25                 .getResultList();
26         } finally {
27             em.close();
28         }
29     }
30
31     public List<Producto> buscarPorNombre(String
  nombre) {
32         EntityManager em = emf.createEntityManager();
33         try {
34             return em.createQuery(
```

```
35         "SELECT p FROM Producto p
        WHERE p.nombre = LOWER(:nombre)",
36         Producto.class
37     ).setParameter("nombre", nombre)
38     .getResultList();
39     } finally {
40         em.close();
41     }
42 }
43
44 }
45
```

```
1 package com.tp.jp.a.repository;
2
3 import com.tp.jp.a.model.entities.Base;
4 import com.tp.jp.a.model.entities.Categoria;
5
6
7
8 public class CategoriaRepository extends
  BaseRepository<Categoria> {
9
10     public CategoriaRepository() {
11         super(Categoria.class);
12     }
13 }
```